# University of Asia Pacific

## Department of Computer Science and Engineering

### Course Code : CSE 450 (VLSI Lab)

### Final Project | Spring 2026

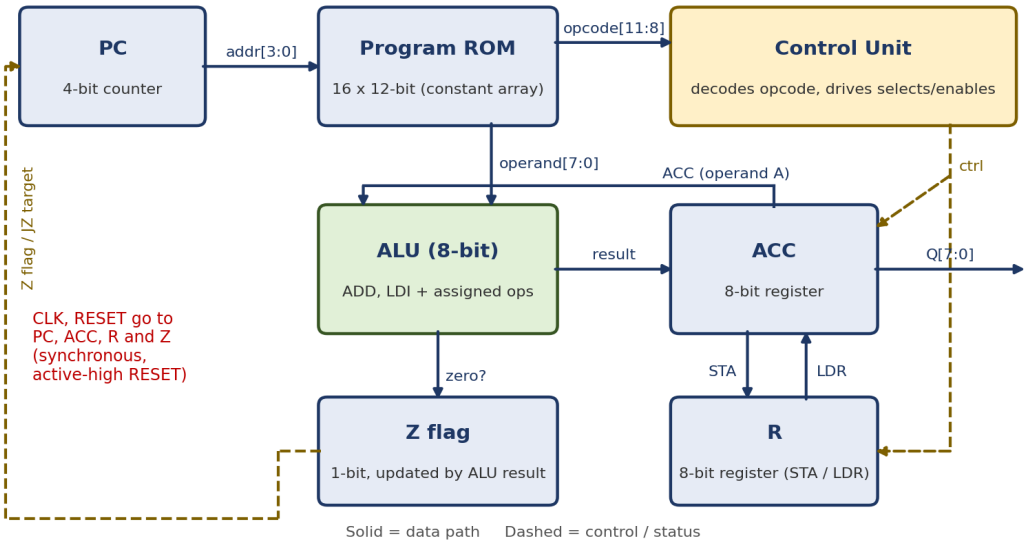
| | | | |
|---|---|---|---|
| **Total Marks** | 50 ( Viva → 30, Report & VrC → 10, Presentation → 10 ) | | |
| **Student Name** | Afiatun Jannat | **ID** | 22201065 |

# Final Project: Mini 8-bit Accumulator CPU

**Objective.** Design and implement a small single-cycle 8-bit accumulator CPU in VHDL. The CPU supports only five instructions: two are mandatory for every student, and the remaining three are assigned to you according to your Student ID (see the “Instruction Set Assignment” page: the last digit of your Student ID decides your set). You must also verify the design with a testbench that runs the program given for your set in Appendix B.

## 1. System Architecture

The CPU is built from the blocks shown below. Every sequential block (PC, ACC, R, Z) is clocked on the rising edge of CLK and cleared by a synchronous, active-high RESET.




| Block | Description |
|---|---|
| **PC** | 4-bit program counter. Holds the address of the instruction being executed. After every instruction it becomes PC + 1, except when JZ is taken. |
| **Program ROM** | 16 locations x 12 bits, written as a constant array in VHDL. It is read combinationally using PC (no clock needed). Unused locations contain 0x000 (NOP). |
| **Control Unit** | Decodes opcode[11:8] and generates the ALU function select and the load/enable signals for ACC, R, Z and PC. |
| **ALU** | 8-bit combinational unit working on ACC and the 8-bit operand. Only the operations of your own instruction set are required. |
| **ACC, R, Z** | ACC: 8-bit accumulator, also the CPU output Q[7:0]. R: 8-bit holding register (only if your set has STA / LDR). Z: 1-bit flag, 1 when the value written to ACC is 0x00. |

## 2. Instruction Format

Each instruction is 12 bits: **instr[11:8] = opcode** (4 bits) and **instr[7:0] = operand** (8-bit immediate). Every instruction takes exactly one clock cycle (fetch, decode and execute happen in the same cycle, the result is written at the rising edge).

**Mandatory instructions (all students):**

| Mnemonic | Opcode | Operation | Flags |
|---|---|---|---|
| **LDI imm** | 0001 | ACC ← imm | Z updated |
| **ADD imm** | 0010 | ACC ← ACC + imm (8-bit, carry discarded) | Z updated |

**Instruction pool (you implement only the three assigned to you):**

| Mnemonic | Opcode | Operation | Flags |
|---|---|---|---|
| **SUB imm** | 0011 | ACC ← ACC − imm (modulo 256) | Z updated |
| **AND imm** | 0100 | ACC ← ACC AND imm | Z updated |
| **OR imm** | 0101 | ACC ← ACC OR imm | Z updated |
| **XOR imm** | 0110 | ACC ← ACC XOR imm | Z updated |
| **NOT** | 0111 | ACC ← NOT ACC (operand ignored) | Z updated |
| **SHL** | 1000 | ACC ← ACC shifted left by 1, 0 enters LSB (operand ignored) | Z updated |
| **SHR** | 1001 | ACC ← ACC shifted right by 1, 0 enters MSB (operand ignored) | Z updated |
| **STA** | 1010 | R ← ACC (operand ignored, ACC unchanged) | Z unchanged |
| **LDR** | 1011 | ACC ← R (operand ignored) | Z updated |
| **JZ addr** | 1100 | If Z = 1 then PC ← operand[3:0], else PC ← PC + 1 | Z unchanged |

**Note:** if your set contains LDR it also contains STA. Any opcode not in your set (including 0000) behaves as NOP: PC ← PC + 1 and nothing else changes.

## 3. Reset Behaviour

**RESET = 1 at a rising edge:** PC ← 0, ACC ← 0x00, R ← 0x00, Z ← 1 (RESET has priority over everything). **RESET = 0 at a rising edge:** execute the instruction at ROM[PC]. Between clock edges all registers keep their values.

## 4. Implementation Requirements

- Use VHDL with IEEE.STD_LOGIC_1164.ALL (IEEE.NUMERIC_STD is allowed for arithmetic).
- Modular design: separate entities for PC, Program ROM, Control Unit, ALU and ACC/R/Z, connected in a top-level CPU entity.
- RESET must be synchronous and active-high. All state changes must occur on the rising edge of CLK.
- The top level must expose CLK, RESET, Q[7:0] (the ACC), the Z flag and, for checking, PC[3:0].
- Implement only the five instructions of your own set. The ROM must hold exactly the Appendix B program of your set; the testbench must generate a clock, apply RESET for one rising edge, and run until the last instruction has executed.

# Instruction Set Assignment

**How to find your set:** take the **last digit of your Student ID** (ID mod 10). That digit is your set number in the table below. Example: ID 22201014 → last digit 4 → Set 4.

Every student implements the two mandatory instructions **LDI** and **ADD** plus the three assigned instructions of his/her set. Total = 5 instructions.

| Last digit of ID | Set | Mandatory | Assigned | Complete instruction set of the CPU |
|---|---|---|---|---|
| 0 | **Set 0** | LDI, ADD | AND, JZ, NOT | LDI, ADD, AND, JZ, NOT |
| 1 | **Set 1** | LDI, ADD | SUB, AND, OR | LDI, ADD, SUB, AND, OR |
| 2 | **Set 2** | LDI, ADD | XOR, NOT, SHL | LDI, ADD, XOR, NOT, SHL |
| 3 | **Set 3** | LDI, ADD | SHR, SUB, XOR | LDI, ADD, SHR, SUB, XOR |
| 4 | **Set 4** | LDI, ADD | STA, LDR, AND | LDI, ADD, STA, LDR, AND |
| 5 | **Set 5** | LDI, ADD | OR, NOT, SHR | LDI, ADD, OR, NOT, SHR |

| Last digit of ID | Set | Mandatory | Assigned | Complete instruction set of the CPU |
|---|---|---|---|---|
| 6 | **Set 6** | LDI, ADD | SUB, JZ, SHL | LDI, ADD, SUB, JZ, SHL |
| 7 | **Set 7** | LDI, ADD | AND, XOR, SHR | LDI, ADD, AND, XOR, SHR |
| 8 | **Set 8** | LDI, ADD | STA, LDR, SUB | LDI, ADD, STA, LDR, SUB |
| 9 | **Set 9** | LDI, ADD | NOT, SHL, JZ | LDI, ADD, NOT, SHL, JZ |

# Appendix B: Required Test Program and Expected Results

Store the program of your set (Appendix B tables are ordered by set number 0–9) in the Program ROM. Apply RESET for one rising edge, then run the CPU for one clock cycle per executed instruction. The ACC, Z, R and next-PC columns show the values after the corresponding rising edge. Addresses that are not listed contain 0x000 (NOP). Operands are in hexadecimal. For NOT, SHL, SHR, STA and LDR the operand field is 0x00.

**ROM encoding:** instr[11:0] = opcode (4 bit) & operand (8 bit). Example: LDI 0x3C = 0001 00111100 = 0x13C.

## Set 0: LDI, ADD, AND, JZ, NOT

**ROM contents:** 0: LDI 0xA5 → 0x1A5 | 1: AND 0x5A → 0x45A | 2: JZ 0x05 → 0xC05 | 3: ADD 0x01 → 0x201 | 4: ADD 0x01 → 0x201 | 5: NOT → 0x700 | 6: AND 0x0F → 0x40F | 7: JZ 0x00 → 0xC00 | 8: ADD 0x01 → 0x201

| Cycle | PC | Instruction executed | ACC (hex) | Z | ACC (binary) | Next PC |
|---|---|---|---|---|---|---|
| 0 | - | RESET = 1 | 0x00 | 1 | 00000000 | 0 |
| 1 | 0 | LDI 0xA5 | 0xA5 | 0 | 10100101 | 1 |
| 2 | 1 | AND 0x5A | 0x00 | 1 | 00000000 | 2 |
| 3 | 2 | JZ 0x05 (taken) | 0x00 | 1 | 00000000 | 5 |
| 4 | 5 | NOT | 0xFF | 0 | 11111111 | 6 |
| 5 | 6 | AND 0x0F | 0x0F | 0 | 00001111 | 7 |
| 6 | 7 | JZ 0x00 (not taken) | 0x0F | 0 | 00001111 | 8 |
| 7 | 8 | ADD 0x01 | 0x10 | 0 | 00010000 | 9 |

## Set 1: LDI, ADD, SUB, AND, OR

**ROM contents:** 0: LDI 0x3C → 0x13C | 1: ADD 0x0F → 0x20F | 2: SUB 0x0B → 0x30B | 3: AND 0xF0 → 0x4F0 | 4: OR 0x0A → 0x50A | 5: SUB 0x4A → 0x34A | 6: SUB 0x01 → 0x301

| Cycle | PC | Instruction executed | ACC (hex) | Z | ACC (binary) | Next PC |
|---|---|---|---|---|---|---|
| 0 | - | RESET = 1 | 0x00 | 1 | 00000000 | 0 |
| 1 | 0 | LDI 0x3C | 0x3C | 0 | 00111100 | 1 |
| 2 | 1 | ADD 0x0F | 0x4B | 0 | 01001011 | 2 |
| 3 | 2 | SUB 0x0B | 0x40 | 0 | 01000000 | 3 |
| 4 | 3 | AND 0xF0 | 0x40 | 0 | 01000000 | 4 |
| 5 | 4 | OR 0x0A | 0x4A | 0 | 01001010 | 5 |
| 6 | 5 | SUB 0x4A | 0x00 | 1 | 00000000 | 6 |
| 7 | 6 | SUB 0x01 | 0xFF | 0 | 11111111 | 7 |

## Set 2: LDI, ADD, XOR, NOT, SHL

**ROM contents:** 0: LDI 0x55 → 0x155 | 1: XOR 0xFF → 0x6FF | 2: NOT → 0x700 | 3: SHL → 0x800 | 4: ADD 0x56 → 0x256 | 5: XOR 0x81 → 0x681 | 6: SHL → 0x800

| Cycle | PC | Instruction executed | ACC (hex) | Z | ACC (binary) | Next PC |
|---|---|---|---|---|---|---|
| 0 | - | RESET = 1 | 0x00 | 1 | 00000000 | 0 |
| 1 | 0 | LDI 0x55 | 0x55 | 0 | 01010101 | 1 |
| 2 | 1 | XOR 0xFF | 0xAA | 0 | 10101010 | 2 |
| 3 | 2 | NOT | 0x55 | 0 | 01010101 | 3 |
| 4 | 3 | SHL | 0xAA | 0 | 10101010 | 4 |
| 5 | 4 | ADD 0x56 | 0x00 | 1 | 00000000 | 5 |
| 6 | 5 | XOR 0x81 | 0x81 | 0 | 10000001 | 6 |
| 7 | 6 | SHL | 0x02 | 0 | 00000010 | 7 |

## Set 3: LDI, ADD, SHR, SUB, XOR

**ROM contents:** 0: LDI 0x80 → 0x180 | 1: SHR → 0x900 | 2: SUB 0x10 → 0x310 | 3: ADD 0x0F → 0x20F | 4: XOR 0x3F → 0x63F | 5: ADD 0x07 → 0x207 | 6: SHR → 0x900

| Cycle | PC | Instruction executed | ACC (hex) | Z | ACC (binary) | Next PC |
|---|---|---|---|---|---|---|
| 0 | - | RESET = 1 | 0x00 | 1 | 00000000 | 0 |
| 1 | 0 | LDI 0x80 | 0x80 | 0 | 10000000 | 1 |
| 2 | 1 | SHR | 0x40 | 0 | 01000000 | 2 |
| 3 | 2 | SUB 0x10 | 0x30 | 0 | 00110000 | 3 |
| 4 | 3 | ADD 0x0F | 0x3F | 0 | 00111111 | 4 |
| 5 | 4 | XOR 0x3F | 0x00 | 1 | 00000000 | 5 |
| 6 | 5 | ADD 0x07 | 0x07 | 0 | 00000111 | 6 |
| 7 | 6 | SHR | 0x03 | 0 | 00000011 | 7 |

## Set 4: LDI, ADD, STA, LDR, AND

**ROM contents:** 0: LDI 0x1E → 0x11E | 1: STA → 0xA00 | 2: ADD 0x22 → 0x222 | 3: AND 0x60 → 0x460 | 4: LDR → 0xB00 | 5: ADD 0x01 → 0x201 | 6: AND 0x0F → 0x40F | 7: STA → 0xA00 | 8: LDI 0x00 → 0x100 | 9: LDR → 0xB00

| Cycle | PC | Instruction executed | ACC (hex) | Z | R (hex) | ACC (bin) | Next PC |
|---|---|---|---|---|---|---|---|
| 0 | - | RESET = 1 | 0x00 | 1 | 0x00 | 00000000 | 0 |
| 1 | 0 | LDI 0x1E | 0x1E | 0 | 0x00 | 00011110 | 1 |
| 2 | 1 | STA | 0x1E | 0 | 0x1E | 00011110 | 2 |
| 3 | 2 | ADD 0x22 | 0x40 | 0 | 0x1E | 01000000 | 3 |
| 4 | 3 | AND 0x60 | 0x40 | 0 | 0x1E | 01000000 | 4 |
| 5 | 4 | LDR | 0x1E | 0 | 0x1E | 00011110 | 5 |
| 6 | 5 | ADD 0x01 | 0x1F | 0 | 0x1E | 00011111 | 6 |
| 7 | 6 | AND 0x0F | 0x0F | 0 | 0x1E | 00001111 | 7 |
| 8 | 7 | STA | 0x0F | 0 | 0x0F | 00001111 | 8 |
| 9 | 8 | LDI 0x00 | 0x00 | 1 | 0x0F | 00000000 | 9 |
| 10 | 9 | LDR | 0x0F | 0 | 0x0F | 00001111 | 10 |

## Set 5: LDI, ADD, OR, NOT, SHR

**ROM contents:** 0: LDI 0x0F → 0x10F | 1: OR 0xA0 → 0x5A0 | 2: NOT → 0x700 | 3: SHR → 0x900 | 4: ADD 0xD8 → 0x2D8 | 5: OR 0x81 → 0x581 | 6: SHR → 0x900

| Cycle | PC | Instruction executed | ACC (hex) | Z | ACC (binary) | Next PC |
|---|---|---|---|---|---|---|
| 0 | - | RESET = 1 | 0x00 | 1 | 00000000 | 0 |
| 1 | 0 | LDI 0x0F | 0x0F | 0 | 00001111 | 1 |
| 2 | 1 | OR 0xA0 | 0xAF | 0 | 10101111 | 2 |
| 3 | 2 | NOT | 0x50 | 0 | 01010000 | 3 |
| 4 | 3 | SHR | 0x28 | 0 | 00101000 | 4 |
| 5 | 4 | ADD 0xD8 | 0x00 | 1 | 00000000 | 5 |
| 6 | 5 | OR 0x81 | 0x81 | 0 | 10000001 | 6 |
| 7 | 6 | SHR | 0x40 | 0 | 01000000 | 7 |

## Set 6: LDI, ADD, SUB, JZ, SHL

**ROM contents:** 0: LDI 0x02 → 0x102 | 1: SUB 0x01 → 0x301 | 2: JZ 0x07 → 0xC07 | 3: SUB 0x01 → 0x301 | 4: JZ 0x07 → 0xC07 | 5: ADD 0xAA → 0x2AA | 6: ADD 0xAA → 0x2AA | 7: LDI 0x21 → 0x121 | 8: SHL → 0x800 | 9: ADD 0x01 → 0x201

| Cycle | PC | Instruction executed | ACC (hex) | Z | ACC (binary) | Next PC |
|---|---|---|---|---|---|---|
| 0 | - | RESET = 1 | 0x00 | 1 | 00000000 | 0 |
| 1 | 0 | LDI 0x02 | 0x02 | 0 | 00000010 | 1 |
| 2 | 1 | SUB 0x01 | 0x01 | 0 | 00000001 | 2 |
| 3 | 2 | JZ 0x07 (not taken) | 0x01 | 0 | 00000001 | 3 |
| 4 | 3 | SUB 0x01 | 0x00 | 1 | 00000000 | 4 |
| 5 | 4 | JZ 0x07 (taken) | 0x00 | 1 | 00000000 | 7 |
| 6 | 7 | LDI 0x21 | 0x21 | 0 | 00100001 | 8 |
| 7 | 8 | SHL | 0x42 | 0 | 01000010 | 9 |
| 8 | 9 | ADD 0x01 | 0x43 | 0 | 01000011 | 10 |

### Set 7: LDI, ADD, AND, XOR, SHR

**ROM contents:** 0: LDI 0xF3 → 0x1F3 | 1: AND 0x3C → 0x43C | 2: XOR 0x11 → 0x611 | 3: SHR → 0x900 | 4: ADD 0x30 → 0x230 | 5: XOR 0x40 → 0x640

| Cycle | PC | Instruction executed | ACC (hex) | Z | ACC (binary) | Next PC |
|---|---|---|---|---|---|---|
| 0 | - | RESET = 1 | 0x00 | 1 | 00000000 | 0 |
| 1 | 0 | LDI 0xF3 | 0xF3 | 0 | 11110011 | 1 |
| 2 | 1 | AND 0x3C | 0x30 | 0 | 00110000 | 2 |
| 3 | 2 | XOR 0x11 | 0x21 | 0 | 00100001 | 3 |
| 4 | 3 | SHR | 0x10 | 0 | 00010000 | 4 |
| 5 | 4 | ADD 0x30 | 0x40 | 0 | 01000000 | 5 |
| 6 | 5 | XOR 0x40 | 0x00 | 1 | 00000000 | 6 |

### Set 8: LDI, ADD, STA, LDR, SUB

**ROM contents:** 0: LDI 0x64 → 0x164 | 1: STA → 0xA00 | 2: SUB 0x14 → 0x314 | 3: ADD 0x0A → 0x20A | 4: STA → 0xA00 | 5: SUB 0x5A → 0x35A | 6: LDR → 0xB00 | 7: SUB 0x60 → 0x360

| Cycle | PC | Instruction executed | ACC (hex) | Z | R (hex) | ACC (bin) | Next PC |
|---|---|---|---|---|---|---|---|
| 0 | - | RESET = 1 | 0x00 | 1 | 0x00 | 00000000 | 0 |
| 1 | 0 | LDI 0x64 | 0x64 | 0 | 0x00 | 01100100 | 1 |
| 2 | 1 | STA | 0x64 | 0 | 0x64 | 01100100 | 2 |
| 3 | 2 | SUB 0x14 | 0x50 | 0 | 0x64 | 01010000 | 3 |
| 4 | 3 | ADD 0x0A | 0x5A | 0 | 0x64 | 01011010 | 4 |
| 5 | 4 | STA | 0x5A | 0 | 0x5A | 01011010 | 5 |
| 6 | 5 | SUB 0x5A | 0x00 | 1 | 0x5A | 00000000 | 6 |
| 7 | 6 | LDR | 0x5A | 0 | 0x5A | 01011010 | 7 |
| 8 | 7 | SUB 0x60 | 0xFA | 0 | 0x5A | 11111010 | 8 |

### Set 9: LDI, ADD, NOT, SHL, JZ

**ROM contents:** 0: LDI 0xFF → 0x1FF | 1: NOT → 0x700 | 2: JZ 0x05 → 0xC05 | 3: ADD 0x77 → 0x277 | 4: ADD 0x77 → 0x277 | 5: ADD 0x11 → 0x211 | 6: SHL → 0x800 | 7: JZ 0x00 → 0xC00 | 8: SHL → 0x800

| Cycle | PC | Instruction executed | ACC (hex) | Z | ACC (binary) | Next PC |
|---|---|---|---|---|---|---|
| 0 | - | RESET = 1 | 0x00 | 1 | 00000000 | 0 |
| 1 | 0 | LDI 0xFF | 0xFF | 0 | 11111111 | 1 |
| 2 | 1 | NOT | 0x00 | 1 | 00000000 | 2 |
| 3 | 2 | JZ 0x05 (taken) | 0x00 | 1 | 00000000 | 5 |
| 4 | 5 | ADD 0x11 | 0x11 | 0 | 00010001 | 6 |
| 5 | 6 | SHL | 0x22 | 0 | 00100010 | 7 |
| 6 | 7 | JZ 0x00 (not taken) | 0x22 | 0 | 00100010 | 8 |
| 7 | 8 | SHL | 0x44 | 0 | 01000100 | 9 |

## Submission Guidelines

Submit your VHDL source files, testbench and a project report within the 2-week period. Include this project description (question, assignment page and Appendix B) at the front of your report and start your own report on a new page after it. The report must include, but is not limited to, the following points.

- Your Student ID, assigned set number and the three assigned instructions (highlighted in yellow color)
- Attach your entity code (all modules), testbench code and output visuals (waveform with PC, ACC, Z and R if used)
- Challenges faced during the experiment, and conclusion

## 1. Student Information and Assigned Instruction Set

| Item | Details |
|---|---|
| University | University of Asia Pacific, Department of Computer Science and Engineering |
| Course | CSE 450 (VLSI Lab), Final Project |
| Project title | Mini 8-bit Accumulator CPU |
| Student Name | Afiatun Jannat |
| Student ID | 22201065 (last digit = 5) |
| Assigned set | **Set 5** |
| Mandatory instructions | LDI, ADD |
| Assigned instructions | **OR, NOT, SHR** |
| Complete instruction set | LDI, ADD, **OR**, **NOT**, **SHR** |
| Language / tools | VHDL; Xilinx ISE VHDL; Xilinx ISE (simulation environment) |

The last digit of the Student ID (22201065 mod 10 = 5) selects Set 5. The CPU implements five instructions, all single-cycle with a 12-bit format (opcode[11:8] and operand[7:0]). The three instructions assigned to Set 5 (OR, NOT, SHR) must be highlighted in yellow in the final Word/PDF copy, as required by the submission guidelines.

**Table 1:** Instruction set implemented by this CPU (Set 5)

| Mnemonic | Opcode | Operation | Flags |
|---|---|---|---|
| LDI imm | 0001 | ACC <- imm | Z updated |
| ADD imm | 0010 | ACC <- ACC + imm (8-bit, carry discarded) | Z updated |
| **OR imm** | 0101 | ACC <- ACC OR imm | Z updated |
| **NOT** | 0111 | ACC <- NOT ACC (operand ignored) | Z updated |
| **SHR** | 1001 | ACC <- ACC shifted right by 1, 0 enters MSB (operand ignored) | Z updated |

Set 5 contains neither STA/LDR nor JZ, so the design has no holding register R and no jump logic. Any opcode outside this set, including 0000, behaves as NOP (PC <- PC + 1, nothing else changes).

## 2. Objective

Design and implement a small single-cycle 8-bit accumulator CPU in VHDL using Xilinx ISE. The CPU supports only five instructions: the two mandatory instructions, LDI and ADD, and the three instructions assigned according to my Student ID 22201065 (last digit 5, Set 5): OR, NOT and SHR. The CPU is built from separate modules (program counter, program ROM, control unit, ALU and ACC/Z register block) connected in a top-level entity, with a 12-bit instruction format (4-bit opcode and 8-bit immediate operand). Every instruction is fetched, decoded and executed in one clock cycle, with all state changes on the rising clock edge and a synchronous, active-high RESET. The design is verified with a testbench that applies RESET for one rising edge and runs the Appendix B program for Set 5. The simulated PC, ACC (Q) and Z waveforms are then compared cycle by cycle with the expected results in Appendix B.

## 3. System Architecture

The CPU is built from the blocks shown in the block diagram below. Every sequential block (PC, ACC, Z) is clocked on the rising edge of CLK and cleared by a synchronous, active-high RESET. Data flows from the PC to the Program ROM, which supplies the opcode to the Control Unit and the operand to the ALU. The ALU combines ACC with the operand, and the result is written back to ACC while the Z flag records whether that result is zero.

(Insert the block diagram of the CPU architecture here: PC -> Program ROM -> Control Unit / ALU -> ACC and Z flag, as given in the project description. Since Set 5 has no STA/LDR and no JZ, the register R and the Z/JZ

feedback path to the PC are not used.)

**Table 2:** Description of the blocks used in this design

| Block | Description |
|---|---|
| PC | 4-bit program counter. Holds the address of the instruction being executed. After every instruction it becomes PC + 1. Set 5 has no JZ, so the PC never jumps. |
| Program ROM | 16 locations x 12 bits, written as a constant array in VHDL. It is read combinationally using PC (no clock needed). Unused locations contain 0x000 (NOP). |
| Control Unit | Decodes opcode[11:8] and generates the ALU function select and the write enables for ACC and Z. Opcodes outside Set 5 produce all enables = 0 (NOP). |
| ALU | 8-bit combinational unit working on ACC and the 8-bit operand. Implements LDI, ADD, OR, NOT and SHR only. |
| ACC, Z | ACC: 8-bit accumulator, also the CPU output Q[7:0]. Z: 1-bit flag, 1 when the value written to ACC is 0x00. Register R is not needed because Set 5 has no STA/LDR. |

**Top-level interface (cpu_top):**

| Port | Direction | Width | Description |
|---|---|---|---|
| CLK | in | 1 | System clock, rising-edge triggered |
| RESET | in | 1 | Synchronous, active-high reset |
| Q | out | 8 | Accumulator (ACC) value |

| | | | |
|---|---|---|---|
| Z | out | 1 | Zero flag |
| PC_OUT | out | 4 | Program counter, for checking |

## 4. Instruction Format and Reset Behaviour

**4.1 Instruction format.** Each instruction is 12 bits: instr[11:8] is the 4-bit opcode and instr[7:0] is the 8-bit immediate operand. Every instruction takes exactly one clock cycle: fetch, decode and execute happen in the same cycle and the result is written at the rising clock edge.

| **Bits** | **11 to 8** | **7 to 0** |
|---|---|---|
| Field | Opcode | Operand (8-bit immediate) |

ROM encoding: instr[11:0] = opcode (4 bit) & operand (8 bit). For example, LDI 0x0F = 0001 00001111 = 0x10F.

**4.2 Opcode map for Set 5.**

| **Mnemonic** | **Opcode** | **ALU_SEL** | **ACC write** | **Z write** | **Result** |
|---|---|---|---|---|---|
| LDI imm | 0001 | 000 | 1 | 1 | imm |
| ADD imm | 0010 | 001 | 1 | 1 | ACC + imm (carry discarded) |
| OR imm | 0101 | 010 | 1 | 1 | ACC OR imm |
| NOT | 0111 | 011 | 1 | 1 | NOT ACC |

| | | | | | |
|---|---|---|---|---|---|
| SHR | 1001 | 100 | 1 | 1 | 0 & ACC[7:1] |
| Others (incl. 0000) | other | 000 | 0 | 0 | NOP: PC <- PC + 1 only |

**4.3 Reset behaviour.**

- RESET = 1 at a rising edge: PC <- 0, ACC <- 0x00, Z <- 1. RESET has priority over everything. (R <- 0x00 applies only to sets with STA/LDR, so it is not present in Set 5.)
- RESET = 0 at a rising edge: the instruction at ROM[PC] is executed. Between clock edges all registers keep their values.

**4.4 Implementation requirements satisfied.**

- VHDL with IEEE.STD_LOGIC_1164.ALL; IEEE.NUMERIC_STD.ALL is used only for the adder and counter arithmetic.
- Modular design: separate entities for PC, Program ROM, Control Unit, ALU and ACC/Z, connected in a top-level CPU entity.
- RESET is synchronous and active-high; all state changes occur on the rising edge of CLK.
- The top level exposes CLK, RESET, Q[7:0] (the ACC), the Z flag and PC[3:0].
- Only the five instructions of Set 5 are implemented, and the ROM holds exactly the Appendix B program of Set 5.

## 5. Design Methodology

The design was developed bottom-up in Xilinx ISE (Project Navigator). Each module was written, synthesised with XST and checked before being connected in the top level.

1. Created a new ISE project (mini_cpu) with the VHDL language and ISim as the simulator.
2. Wrote the program counter (pc.vhd), a 4-bit counter with synchronous reset that adds 1 on every rising edge.
3. Wrote the program ROM (prog_rom.vhd), a 16 x 12-bit constant array holding the Set 5 program, read combinationally from the PC.
4. Wrote the ALU (alu.vhd), a combinational block selecting among LDI, ADD, OR, NOT and SHR through a 3-bit ALU_SEL.
5. Wrote the control unit (control_unit.vhd), which decodes the opcode and drives ALU_SEL, ACC_WE and Z_WE; unsupported opcodes give a NOP.
6. Wrote the ACC and Z register block (acc_z.vhd) with synchronous reset (ACC = 0x00, Z = 1) and write enables.
7. Wrote the top level (cpu_top.vhd), instantiating the five modules with port maps.
8. Wrote testbenches for each module and a full-CPU testbench (cpu_tb.vhd) that runs the Appendix B program.
9. Ran behavioural simulation in ISim and compared PC, Q and Z with the expected Appendix B values.
10. Checked the synthesis result with the RTL Schematic viewer.

**Module descriptions.**

| Module | File | Inputs | Outputs | Function |
|---|---|---|---|---|
| Program counter | pc.vhd | CLK, RESET | PC_OUT[3:0] | Resets to 0, otherwise increments each rising edge |
| Program ROM | prog_rom.vhd | ADDR[3:0] | INSTR[11:0] | Returns the stored instruction for the given address |
| ALU | alu.vhd | ACC[7:0], IMM[7:0], ALU_SEL[2:0] | RESULT[7:0] | Performs LDI, ADD, OR, NOT, SHR |
| Control unit | control_unit.vhd | OPCODE[3:0] | ALU_SEL[2:0], ACC_WE, Z_WE | Decodes the opcode into control signals |
| ACC and Z | acc_z.vhd | CLK, RESET, ACC_WE, Z_WE, RESULT[7:0] | ACC_OUT[7:0], Z_OUT | Stores ACC and the zero flag |
| Top level | cpu_top.vhd | CLK, RESET | Q[7:0], Z, PC_OUT[3:0] | Connects all modules |

**Synthesis check.** The RTL schematic of the ALU generated by XST shows an adder (ADD), an or2 gate (OR), an inverter (NOT), plain wiring with a ground constant for the MSB (SHR) and a multiplexer driven by ALU_SEL(2:0), which matches the VHDL description. (Insert the ALU RTL schematic screenshot here, and optionally the RTL schematics of the other modules.)

## 6. VHDL Source Code (Entity Code of All Modules)

**6.1 Program counter (pc.vhd).** A 4-bit counter. On a rising edge it resets to 0 when RESET = 1, otherwise it increments by 1. No jump logic is needed because Set 5 has no JZ.

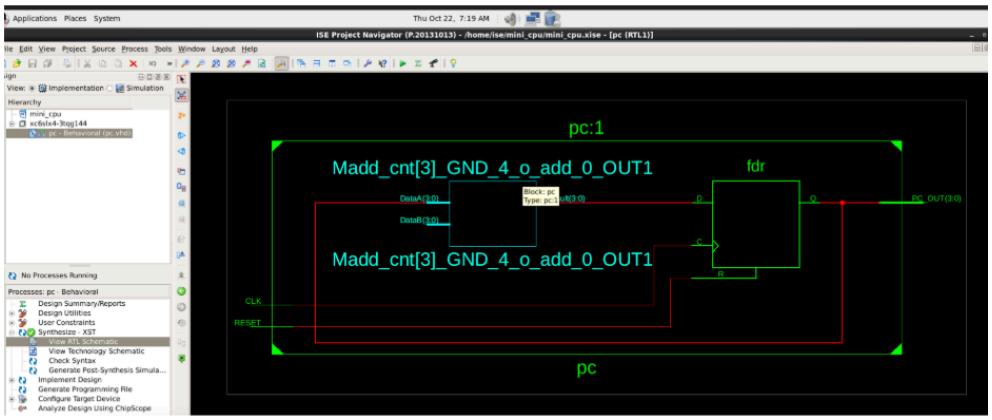
```
library IEEE;
```

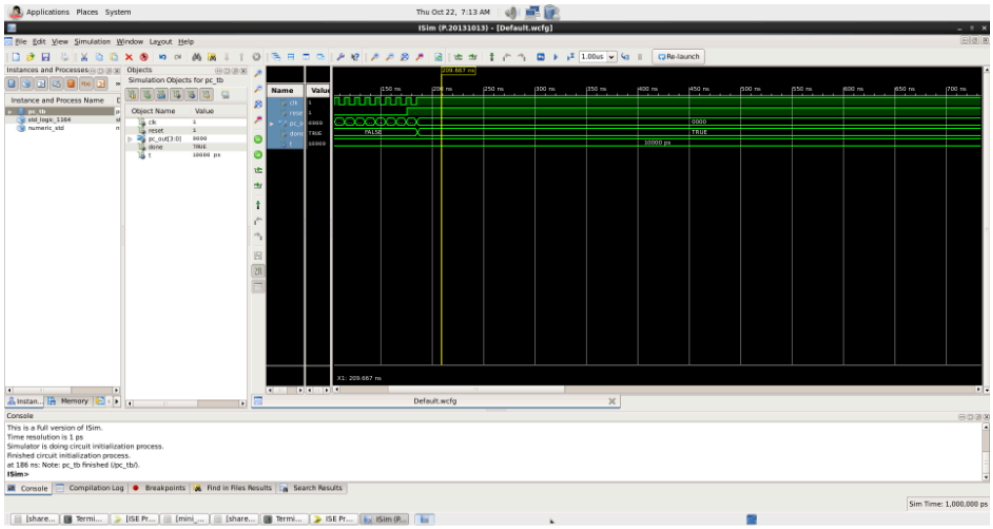
```
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity pc is
    Port ( CLK    : in  STD_LOGIC;
           RESET  : in  STD_LOGIC;
           PC_OUT : out STD_LOGIC_VECTOR(3 downto 0));
end pc;

architecture Behavioral of pc is
    signal cnt : unsigned(3 downto 0) := (others => '0');
begin
    process(CLK)
    begin
        if rising_edge(CLK) then
            if RESET = '1' then
                cnt <= (others => '0');
            else
                cnt <= cnt + 1;
            end if;
        end if;
    end process;

    PC_OUT <= std_logic_vector(cnt);
end Behavioral;
```



**6.2 Program ROM (prog_rom.vhd).** A 16 x 12-bit constant array holding the Appendix B Set 5 program. It has no clock and is read combinationally from the address. Unused locations hold 0x000 (NOP).

```
library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

use IEEE.NUMERIC_STD.ALL;



entity prog_rom is

    Port ( ADDR  : in  STD_LOGIC_VECTOR(3 downto 0);

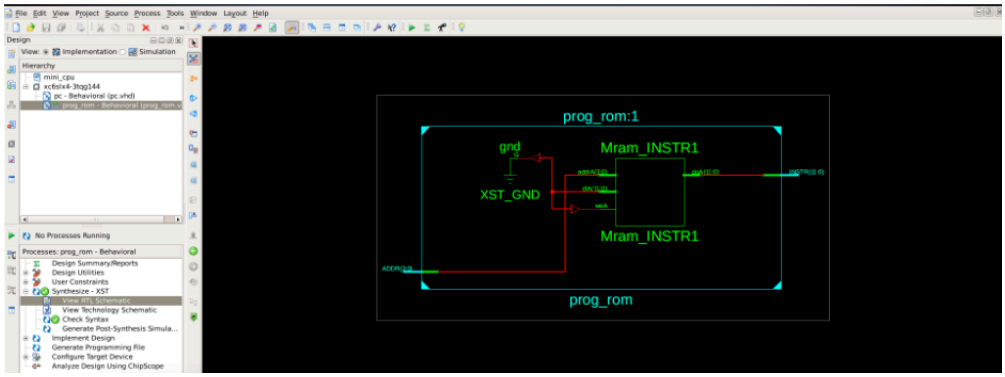
           INSTR : out STD_LOGIC_VECTOR(11 downto 0));

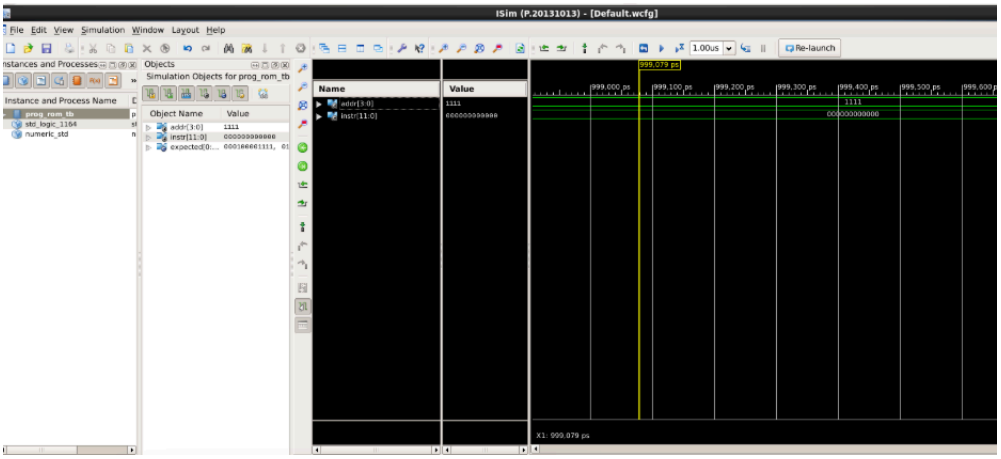
end prog_rom;



architecture Behavioral of prog_rom is
```

```
    type rom_type is array (0 to 15) of STD_LOGIC_VECTOR(11 downto 0);
    constant ROM : rom_type := (
        0 => x"10F",   -- LDI 0x0F
        1 => x"5A0",   -- OR  0xA0
        2 => x"700",   -- NOT
        3 => x"900",   -- SHR
        4 => x"2D8",   -- ADD 0xD8
        5 => x"581",   -- OR  0x81
        6 => x"900",   -- SHR
        others => x"000"  -- NOP
    );
begin
    INSTR <= ROM(to_integer(unsigned(ADDR)));
end Behavioral;
```



**6.3 ALU (alu.vhd).** A combinational block. It takes ACC, the 8-bit operand and a 3-bit select, and outputs the result. The ADD carry is discarded by the 8-bit result width, and SHR shifts right with 0 entering the MSB.

```
library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

use IEEE.NUMERIC_STD.ALL;


entity alu is

   Port ( ACC     : in  STD_LOGIC_VECTOR(7 downto 0);

          IMM     : in  STD_LOGIC_VECTOR(7 downto 0);

          ALU_SEL : in  STD_LOGIC_VECTOR(2 downto 0);

          RESULT  : out STD_LOGIC_VECTOR(7 downto 0));

end alu;


architecture Behavioral of alu is

begin

   process(ACC, IMM, ALU_SEL)

   begin

      case ALU_SEL is

         when "000" => RESULT <= IMM;                              -- LDI

         when "001" => RESULT <= std_logic_vector(unsigned(ACC) + unsigned(IMM)); -- ADD (carry dropped)

         when "010" => RESULT <= ACC or IMM;                       -- OR

         when "011" => RESULT <= not ACC;                          -- NOT

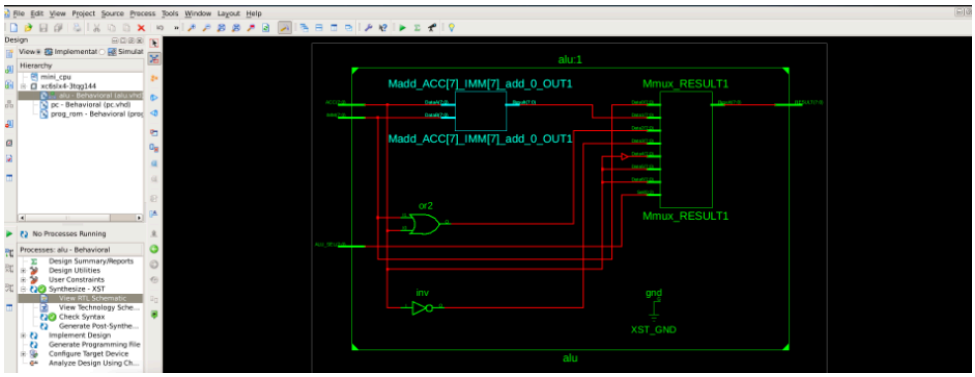
         when "100" => RESULT <= '0' & ACC(7 downto 1);            -- SHR (0 enters MSB)

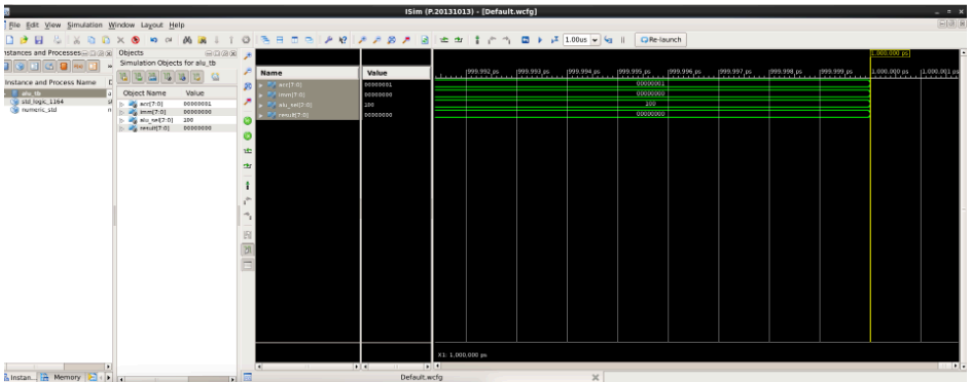
         when others => RESULT <= ACC;

      end case;

   end process;

end Behavioral;
```



**6.4 Control unit (control_unit.vhd).** Decodes instr[11:8] and produces the ALU select and the write enables. Any opcode not in Set 5, including 0000, leaves all enables at 0, which is a NOP.

```
library IEEE;

use IEEE.STD_LOGIC_1164.ALL;


entity control_unit is

    Port ( OPCODE  : in  STD_LOGIC_VECTOR(3 downto 0);

           ALU_SEL : out STD_LOGIC_VECTOR(2 downto 0);

           ACC_WE  : out STD_LOGIC;

           Z_WE    : out STD_LOGIC);

end control_unit;


architecture Behavioral of control_unit is

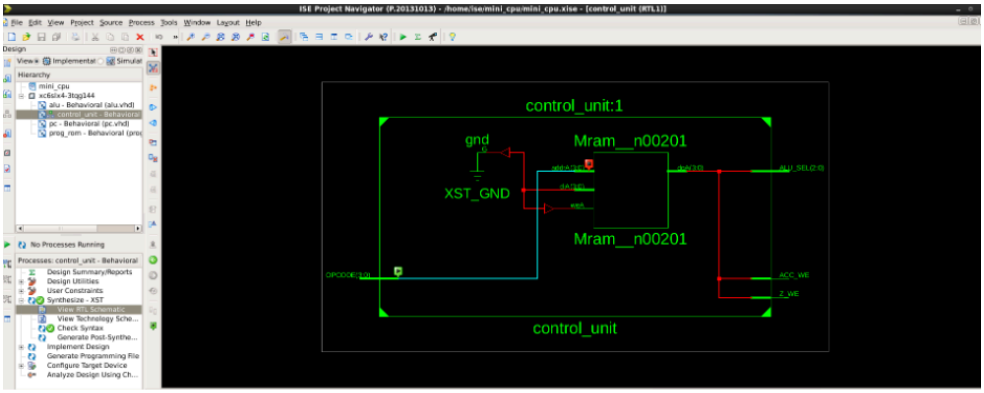
begin

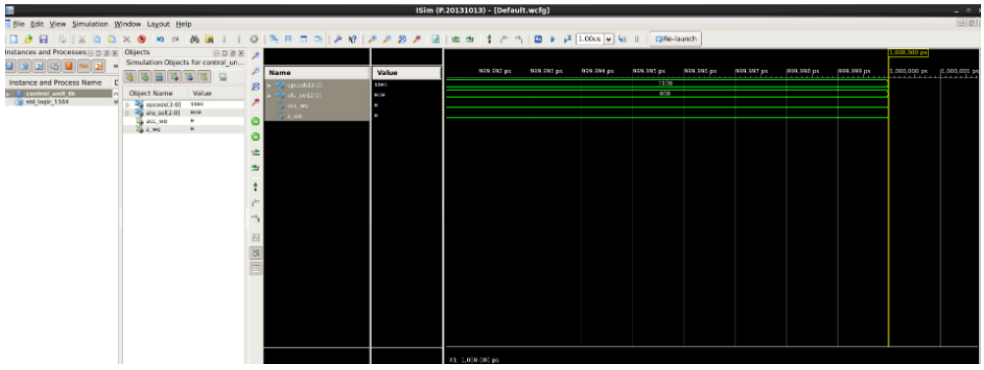
    process(OPCODE)

    begin
```

```
        -- defaults = NOP
        ALU_SEL <= "000";
        ACC_WE  <= '0';
        Z_WE    <= '0';

        case OPCODE is
            when "0001" => ALU_SEL <= "000"; ACC_WE <= '1'; Z_WE <= '1'; -- LDI
            when "0010" => ALU_SEL <= "001"; ACC_WE <= '1'; Z_WE <= '1'; -- ADD
            when "0101" => ALU_SEL <= "010"; ACC_WE <= '1'; Z_WE <= '1'; -- OR
            when "0111" => ALU_SEL <= "011"; ACC_WE <= '1'; Z_WE <= '1'; -- NOT
            when "1001" => ALU_SEL <= "100"; ACC_WE <= '1'; Z_WE <= '1'; -- SHR
            when others => null;                                  -- NOP
        end case;
    end process;
end Behavioral;
```



**6.5 ACC and Z register block (acc_z.vhd).** Holds the accumulator and the zero flag. On reset, ACC = 0x00 and Z = 1. Z becomes 1 when the value written to ACC is 0x00.

```
library IEEE;

use IEEE.STD_LOGIC_1164.ALL;



entity acc_z is

    Port ( CLK     : in  STD_LOGIC;

           RESET   : in  STD_LOGIC;

           ACC_WE  : in  STD_LOGIC;

           Z_WE    : in  STD_LOGIC;

           RESULT  : in  STD_LOGIC_VECTOR(7 downto 0);

           ACC_OUT : out STD_LOGIC_VECTOR(7 downto 0);

           Z_OUT   : out STD_LOGIC);

end acc_z;



architecture Behavioral of acc_z is

    signal acc_reg : STD_LOGIC_VECTOR(7 downto 0) := (others => '0');

    signal z_reg   : STD_LOGIC := '1';

begin

    process(CLK)

    begin

        if rising_edge(CLK) then

            if RESET = '1' then

                acc_reg <= (others => '0');

                z_reg   <= '1';

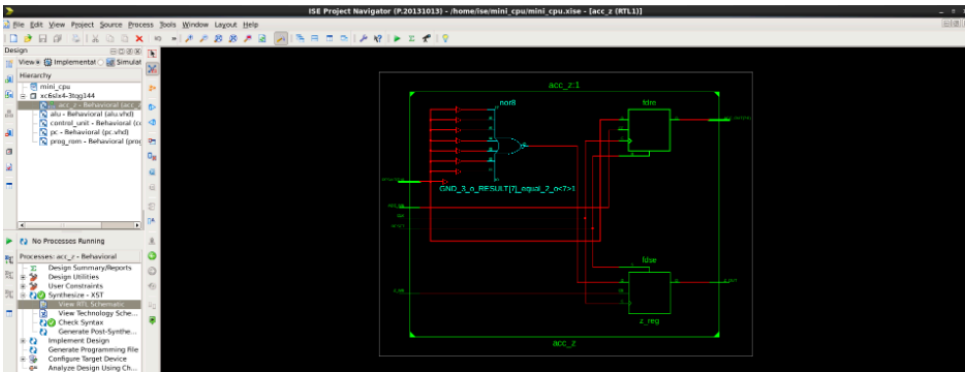
            else

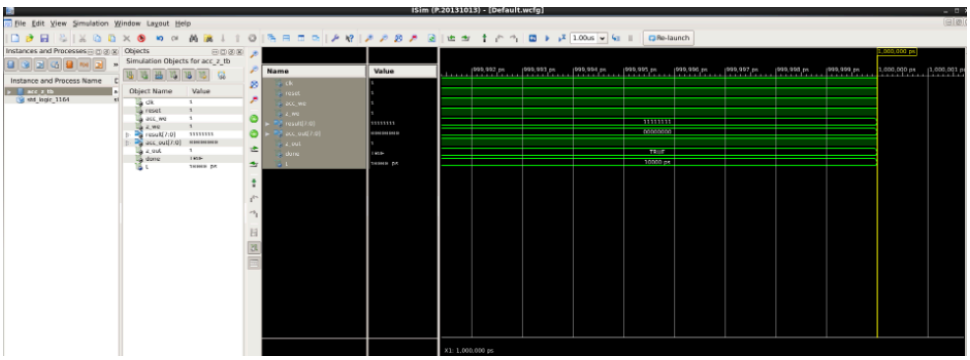
                if ACC_WE = '1' then

                    acc_reg <= RESULT;
```

```
            end if;
            if Z_WE = '1' then
                if RESULT = "00000000" then
                    z_reg <= '1';
                else
                    z_reg <= '0';
                end if;
            end if;
        end if;
    end if;
end process;

ACC_OUT <= acc_reg;
Z_OUT   <= z_reg;
end Behavioral;
```





**6.6 Top-level CPU (cpu_top.vhd).** Connects the PC, ROM, control unit, ALU and ACC/Z block. The program

counter output is named PC_OUT to avoid a clash with the entity name pc.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;


entity cpu_top is
    Port ( CLK    : in  STD_LOGIC;
           RESET  : in  STD_LOGIC;
           Q      : out STD_LOGIC_VECTOR(7 downto 0);
           Z      : out STD_LOGIC;
           PC_OUT : out STD_LOGIC_VECTOR(3 downto 0));
end cpu_top;


architecture Structural of cpu_top is
    signal pc_s    : STD_LOGIC_VECTOR(3 downto 0);
    signal instr   : STD_LOGIC_VECTOR(11 downto 0);
    signal alu_sel : STD_LOGIC_VECTOR(2 downto 0);
    signal acc_we  : STD_LOGIC;
    signal z_we    : STD_LOGIC;
    signal acc_s   : STD_LOGIC_VECTOR(7 downto 0);
    signal result  : STD_LOGIC_VECTOR(7 downto 0);
    signal z_s     : STD_LOGIC;
begin
    u_pc  : entity work.pc
        port map (CLK => CLK, RESET => RESET, PC_OUT => pc_s);

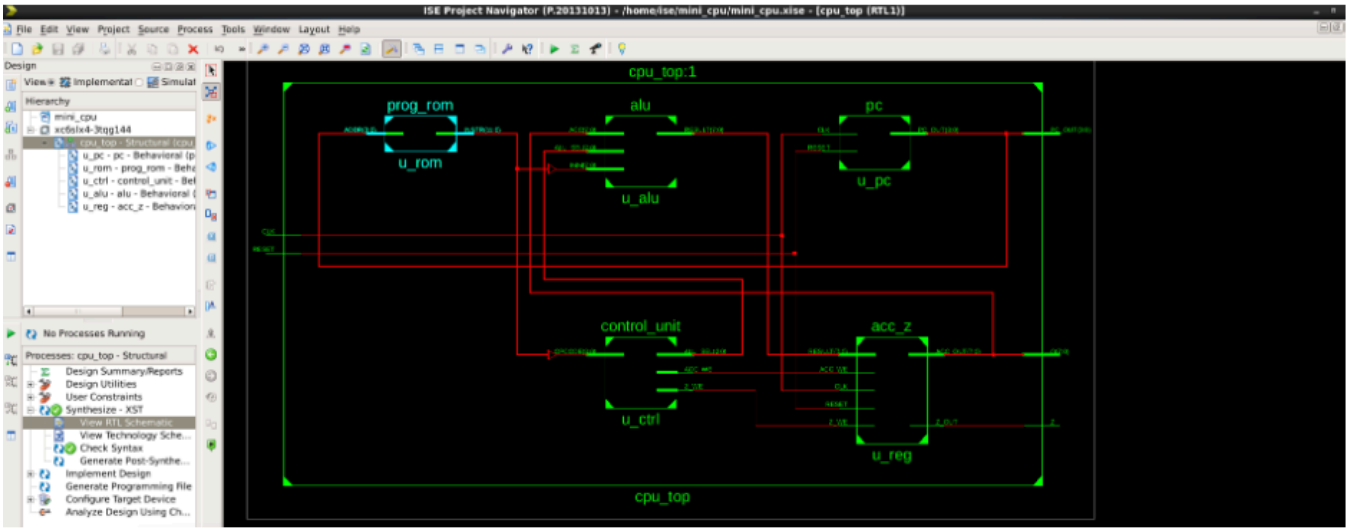

    u_rom : entity work.prog_rom
        port map (ADDR => pc_s, INSTR => instr);
```

```
    u_ctrl: entity work.control_unit
        port map (OPCODE => instr(11 downto 8), ALU_SEL => alu_sel,
                  ACC_WE => acc_we, Z_WE => z_we);

    u_alu : entity work.alu
        port map (ACC => acc_s, IMM => instr(7 downto 0),
                  ALU_SEL => alu_sel, RESULT => result);

    u_reg : entity work.acc_z
        port map (CLK => CLK, RESET => RESET, ACC_WE => acc_we, Z_WE => z_we,
                  RESULT => result, ACC_OUT => acc_s, Z_OUT => z_s);

    Q      <= acc_s;
    Z      <= z_s;
    PC_OUT <= pc_s;
end Structural;
```




## 7. Testbench Code

**7.1 Full CPU testbench (cpu_tb.vhd).** This is the required testbench. It generates a 10 ns clock, applies RESET

for exactly one rising edge, then runs the CPU for seven instruction cycles, which is every instruction of the Set 5 Appendix B program, and finally stops the clock.

```
library IEEE;

use IEEE.STD_LOGIC_1164.ALL;



entity cpu_tb is

end cpu_tb;



architecture Behavioral of cpu_tb is

    signal CLK    : STD_LOGIC := '0';

    signal RESET  : STD_LOGIC := '1';

    signal Q      : STD_LOGIC_VECTOR(7 downto 0);

    signal Z      : STD_LOGIC;

    signal PC_OUT : STD_LOGIC_VECTOR(3 downto 0);

    signal done   : boolean := false;

    constant T    : time := 10 ns;

begin

    uut : entity work.cpu_top

        port map (CLK => CLK, RESET => RESET, Q => Q, Z => Z, PC_OUT => PC_OUT);



    -- clock: rising edges at 5, 15, 25 ... ns

    clk_proc : process

    begin

        while not done loop

            CLK <= '0'; wait for T/2;

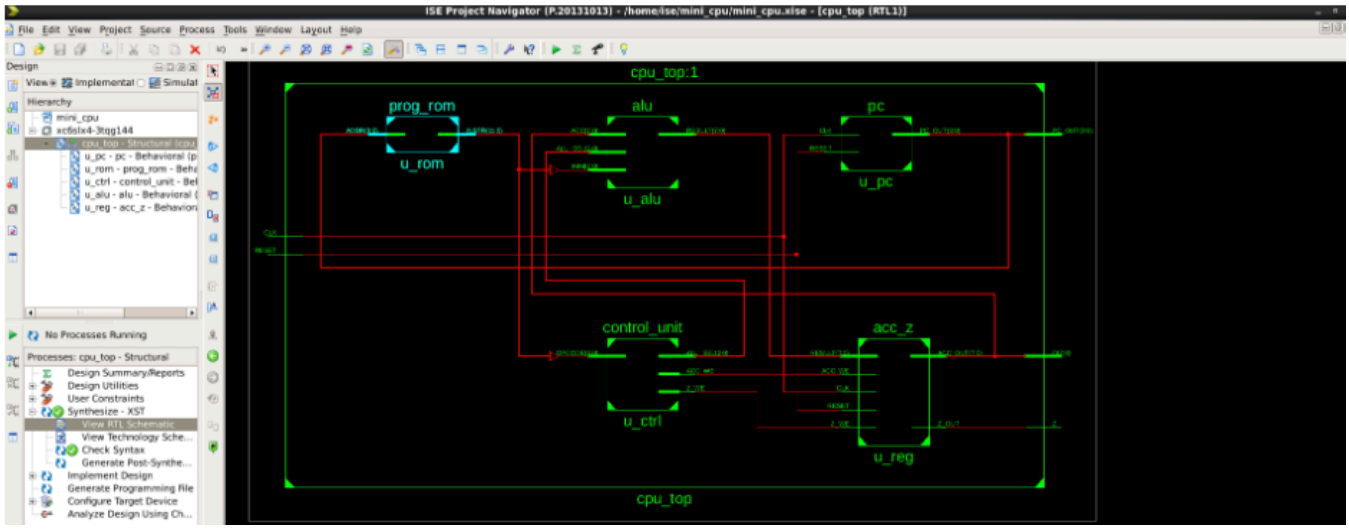
            CLK <= '1'; wait for T/2;

        end loop;

        wait;
```

```
    end process;


    stim : process
    begin
        RESET <= '1';
        wait for T;            -- covers exactly one rising edge (at 5 ns)
        RESET <= '0';
        wait for 7 * T;        -- 7 instructions (edges at 15 ... 75 ns)
        wait for T;            -- small margin
        done <= true;          -- stops the clock
        wait;
    end process;
end Behavioral;
```




**7.2 Module testbenches.** Self-checking testbenches (using assert statements) were also written for each module. Each one reports "finished" in the console when all checks pass.

| Testbench | Module under test | What it verifies |
|---|---|---|
| pc_tb.vhd | pc | Reset to 0, counting 0 to 15, wrap-around to 0, reset |

| | | |
|---|---|---|
| | | during counting |
| prog_rom_tb.vhd | prog_rom | All 16 locations match the Set 5 program; locations 7 to 15 are 0x000 |
| alu_tb.vhd | alu | LDI, ADD with and without overflow, OR, NOT, SHR including lost LSB |
| control_unit_tb.vhd | control_unit | ALU_SEL and enables for the five opcodes; all other opcodes behave as NOP |
| acc_z_tb.vhd | acc_z | Reset values, write, Z flag on zero, hold with enables off, reset priority |

(Paste the complete code of pc_tb, prog_rom_tb, alu_tb, control_unit_tb and acc_z_tb here, exactly as written in your ISE project.)

## 8. Simulation Results and Verification

**8.1 Test program (Appendix B, Set 5).** The program stored in the Program ROM is:

| Address | Instruction | Encoding (hex) |
|---|---|---|
| 0 | LDI 0x0F | 0x10F |
| 1 | OR 0xA0 | 0x5A0 |
| 2 | NOT | 0x700 |
| 3 | SHR | 0x900 |
| 4 | ADD 0xD8 | 0x2D8 |

| | | |
|---|---|---|
| 5 | OR 0x81 | 0x581 |
| 6 | SHR | 0x900 |
| 7 to 15 | (unused) | 0x000 (NOP) |

**8.2 Test conditions.** The clock period is 10 ns, so rising edges occur at 5 ns, 15 ns, 25 ns and so on. RESET = 1 is applied for the first rising edge (5 ns) and then released. Cycle n therefore executes at the rising edge at (5 + 10n) ns, and the seven instructions finish at 75 ns.

**8.3 Expected results.** The table below is the expected result from Appendix B. ACC, Z and next PC are the values after the corresponding rising edge.

| Cycle | Time (ns) | PC | Instruction executed | ACC (hex) | Z | ACC (binary) | Next PC |
|---|---|---|---|---|---|---|---|
| 0 | 5 | - | RESET = 1 | 0x00 | 1 | 00000000 | 0 |
| 1 | 15 | 0 | LDI 0x0F | 0x0F | 0 | 00001111 | 1 |
| 2 | 25 | 1 | OR 0xA0 | 0xAF | 0 | 10101111 | 2 |
| 3 | 35 | 2 | NOT | 0x50 | 0 | 01010000 | 3 |
| 4 | 45 | 3 | SHR | 0x28 | 0 | 00101000 | 4 |
| 5 | 55 | 4 | ADD 0xD8 | 0x00 | 1 | 00000000 | 5 |
| 6 | 65 | 5 | OR 0x81 | 0x81 | 0 | 10000001 | 6 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 7 | 75 | 6 | SHR | 0x40 | 0 | 01000000 | 7 |

**8.4 Explanation of the key cycles.**

- Cycle 1 (LDI): ACC is loaded with the immediate 0x0F and Z = 0 because the value is not zero.
- Cycle 2 (OR): 0x0F OR 0xA0 = 0xAF.
- Cycle 3 (NOT): the bitwise complement of 0xAF (10101111) is 0x50 (01010000).
- Cycle 4 (SHR): 0x50 shifted right by one bit is 0x28, with 0 entering the MSB.
- Cycle 5 (ADD): 0x28 + 0xD8 = 0x100, and the carry is discarded, so ACC = 0x00 and Z becomes 1. This also tests the Z flag and the 8-bit overflow behaviour.
- Cycle 6 (OR): 0x00 OR 0x81 = 0x81, so Z returns to 0.
- Cycle 7 (SHR): 0x81 shifted right by one bit gives 0x40, and the LSB is lost.

**8.5 Simulation waveforms.** Insert the following screenshots from ISim, with Q and PC_OUT set to hexadecimal radix:

- **Figure 1:** Full CPU waveform (cpu_tb) showing CLK, RESET, PC_OUT, Q (ACC) and Z over the whole program run. This is the main required output.
- **Figure 2:** (optional) Zoomed view of the ADD 0xD8 cycle showing ACC = 0x00 and Z = 1.
- **Figure 3 to 7:** (optional) Waveforms of pc_tb, prog_rom_tb, alu_tb, control_unit_tb and acc_z_tb as proof of unit testing.
- **Figure 8:** RTL schematic of the ALU (and optionally of the other modules).

Set 5 has no register R, so the waveform contains PC, ACC (Q) and Z only.

**8.6 Verification summary.** The simulated Q, Z and PC_OUT values were compared cycle by cycle with Table in 8.3. The ACC sequence 00, 0F, AF, 50, 28, 00, 81, 40 and the Z sequence 1, 0, 0, 0, 0, 1, 0, 0 match the Appendix B expected results, and PC counts 0 to 7 without jumps. All module testbenches finished without assertion errors. (Confirm this against your own waveform before submitting; if any value differs, correct the design first.)

## 9. Challenges Faced

- **Tool mismatch:** The project was done in Xilinx ISE (Spartan-6, XST and ISim), whose menus for adding sources and running simulation differ from newer tools. Test benches had to be created through Project, New Source, VHDL Test Bench and linked to the right module, and the hierarchy view switched from Implementation to Simulation.
- **Incomplete hierarchy:** At first only alu, pc and prog_rom appeared in the hierarchy. The remaining modules (control_unit, acc_z, cpu_top) had to be added and cpu_top set as the top module so that all modules were nested under it.
- **Single clock edge for reset:** The specification requires RESET for exactly one rising edge. The testbench clock had to start at 0 with RESET = 1 from time 0, so that the first rising edge at 5 ns captures the reset and the registers do not show undefined values (U or X).
- **Naming clash:** The top-level output port could not be called PC because it clashes with the entity pc, so it was named PC_OUT.
- **Carry and shift behaviour:** ADD had to discard the carry (0x28 + 0xD8 = 0x00 with Z = 1), and SHR had to place a 0 in the MSB. Both were handled with an 8-bit result and the expression '0' & ACC(7 downto 1).
- **NOP handling:** Opcodes outside Set 5, including 0000, had to leave ACC and Z unchanged. This was solved by giving the control unit default outputs of zero for all enabled before decoding.
- **Matching timing to Appendix B:** Values are written at the rising edge, so waveforms had to be read just

after each edge, and the simulation run length had to cover all seven instructions.

- **Unit-level checking:** Individual self-checking testbenches were needed so that an error could be traced to one module instead of only being seen in the final CPU waveform.

## 10. Conclusion

A single-cycle 8-bit accumulator CPU was designed in VHDL and simulated in Xilinx ISE for Set 5 (Student ID 22201065). The CPU implements the mandatory instructions LDI and ADD and the assigned instructions OR, NOT and SHR. It is built from five modules (program counter, program ROM, control unit, ALU, and ACC/Z register block) joined in a top-level cpu_top entity with the ports CLK, RESET, Q[7:0], Z and PC[3:0].The Appendix B program was stored in the ROM and run with a testbench that applies RESET for one rising edge. The resulting ACC, Z and PC values follow the expected results cycle by cycle, including the 8-bit overflow in ADD, the Z flag going to 1 on a zero result and the 0 entering the MSB in SHR. The RTL schematic confirms that the ALU synthesises into an adder, an OR gate, an inverter and a multiplexer, as described in the VHDL.This project gave practical experience in modular VHDL design, synchronous design with a single clock and reset, instruction decoding, testbench writing and waveform-based verification. The design could be extended with further instructions from the pool (such as SUB, AND, XOR, SHL, STA, LDR and JZ), a larger ROM or a wider program counter.

## 11.Github Link

**Github**